

Прогнозирование спроса на велопрокат

Иванов Александр Евгеньевич 424-3

Искусственный интеллект. Алгоритмы машинного обучения на языке Python

Постановка задачи

Capital Bikeshare - сервис
велопроката в Вашингтоне (США)

Оператор хочет точно
прогнозировать почасовой спрос
на велосипеды, при помощи
собранной истории



Тип задачи и метрики

Задача: Регрессия

Целевая переменная: **cnt** - непрерывная (целочисленная) переменная, количество арендованных велосипедов за час

Метрика	Обоснование
RMSE	Основная метрика; сильнее штрафует за крупные ошибки, что важно для пиковых часов
MAE	Легко интерпретируется операционной командой (ошибка в единицах велосипедов)
R^2	Доля объяснённой дисперсии; удобен для сравнения моделей

Ограничения и допущения

- Данные охватывают только два года (2011–2012)
- Данные по одному городу (Вашингтон, DC)
- Внешние события в датасете не отражены

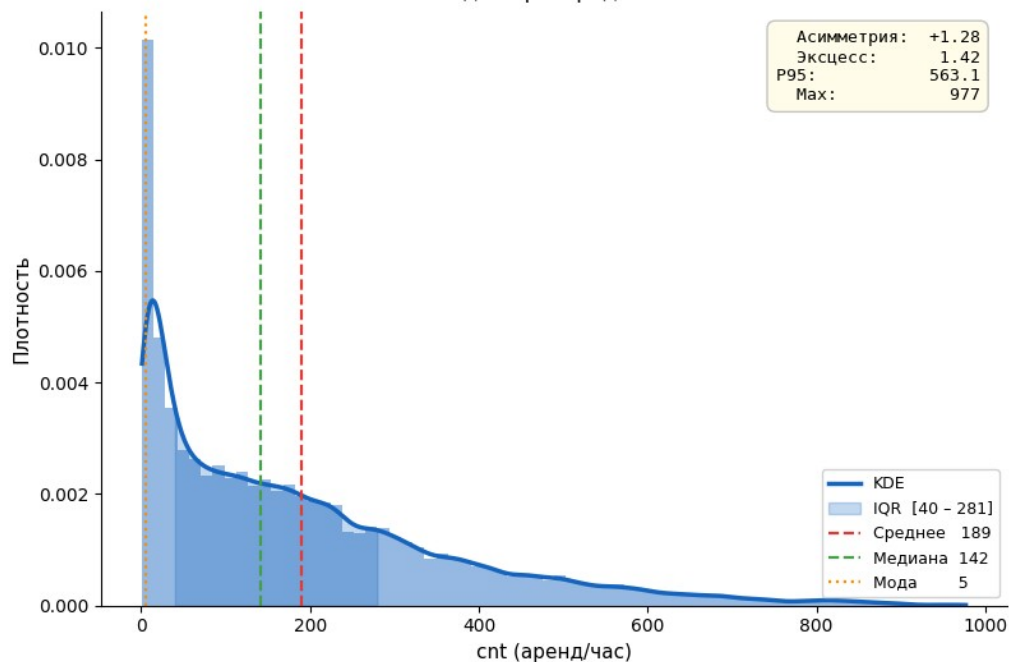


EDA и Feature Engineering

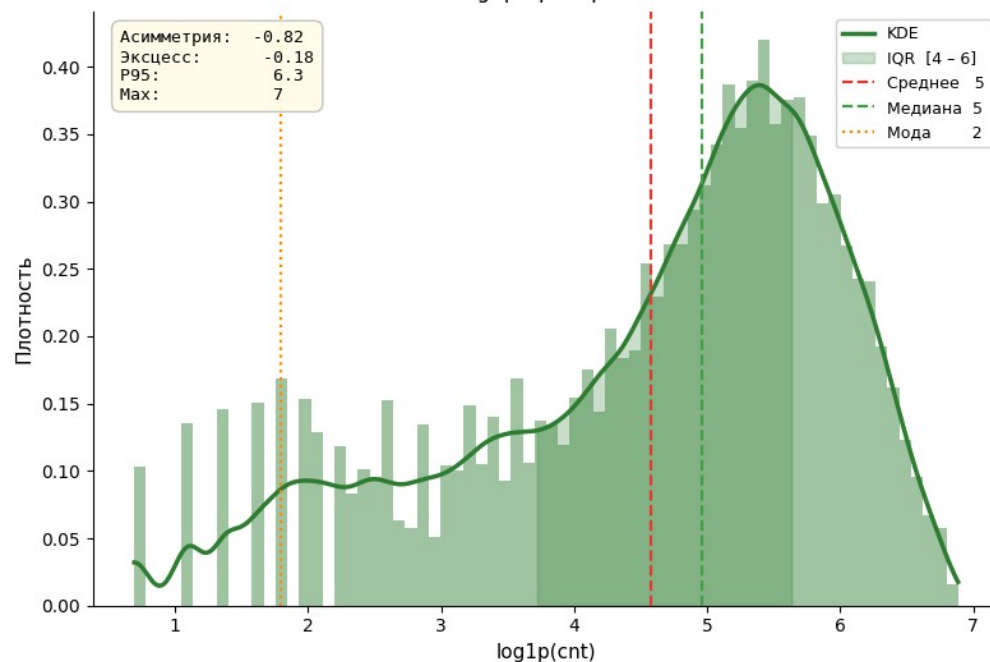
Анализ целевой переменной

Распределение целевой переменной cnt

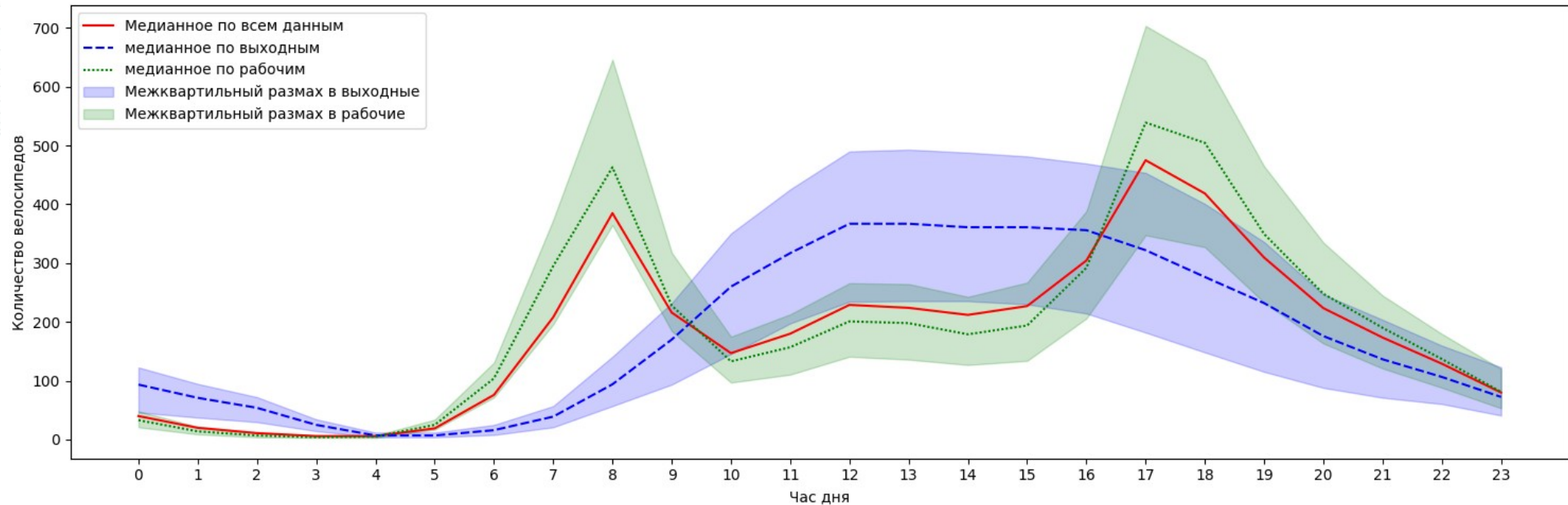
Исходное распределение



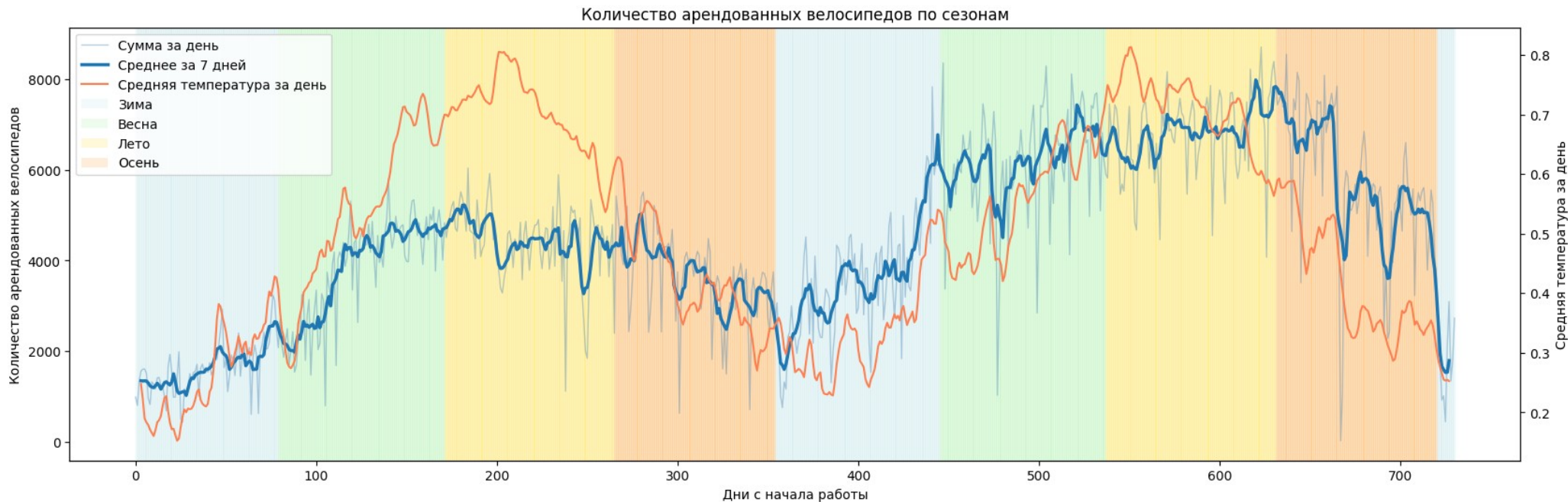
После log1p-преобразования



Анализ количества аренды по времени

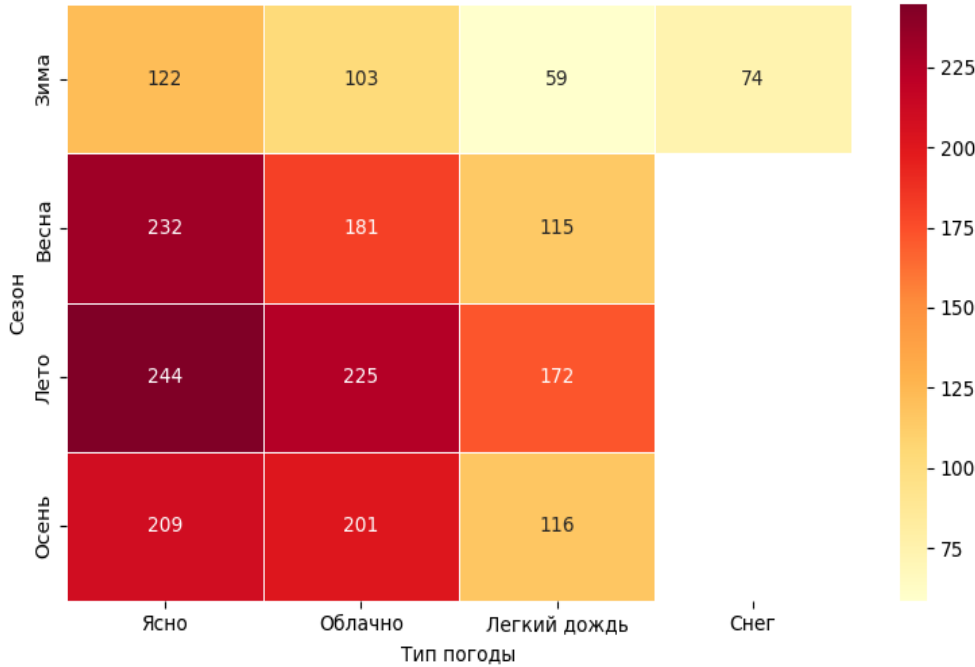


Анализ количества аренды по сезонам

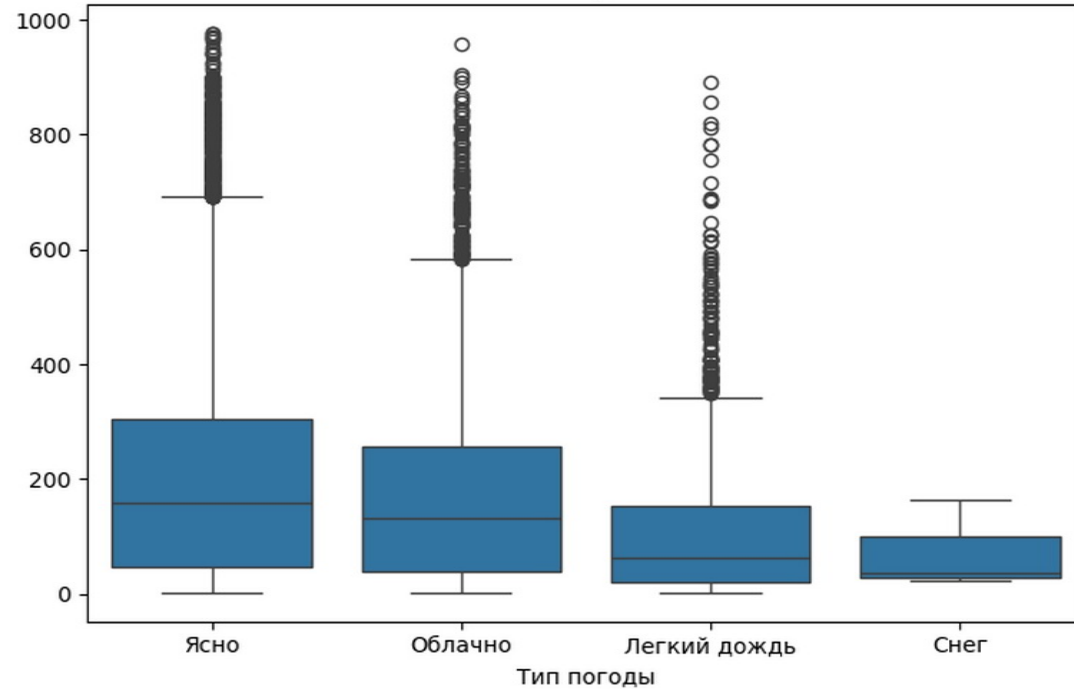


Heatmap по сезонам и погодам

Средний спрос: сезон x тип погоды

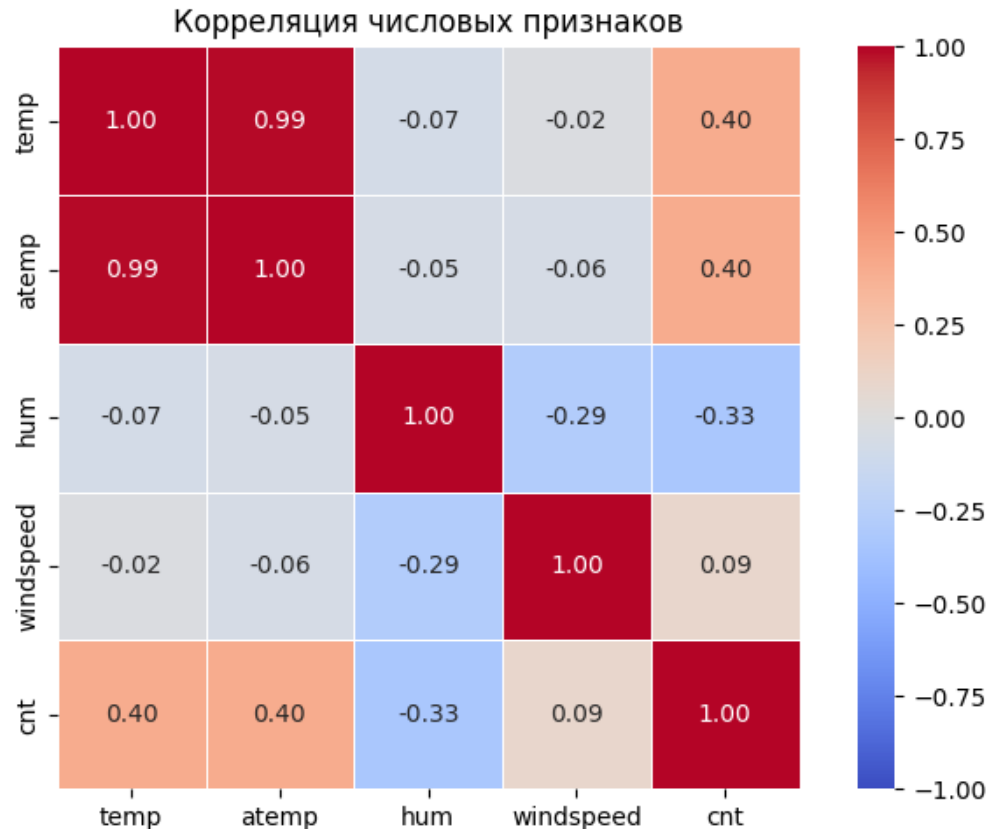


cnt по типу погоды

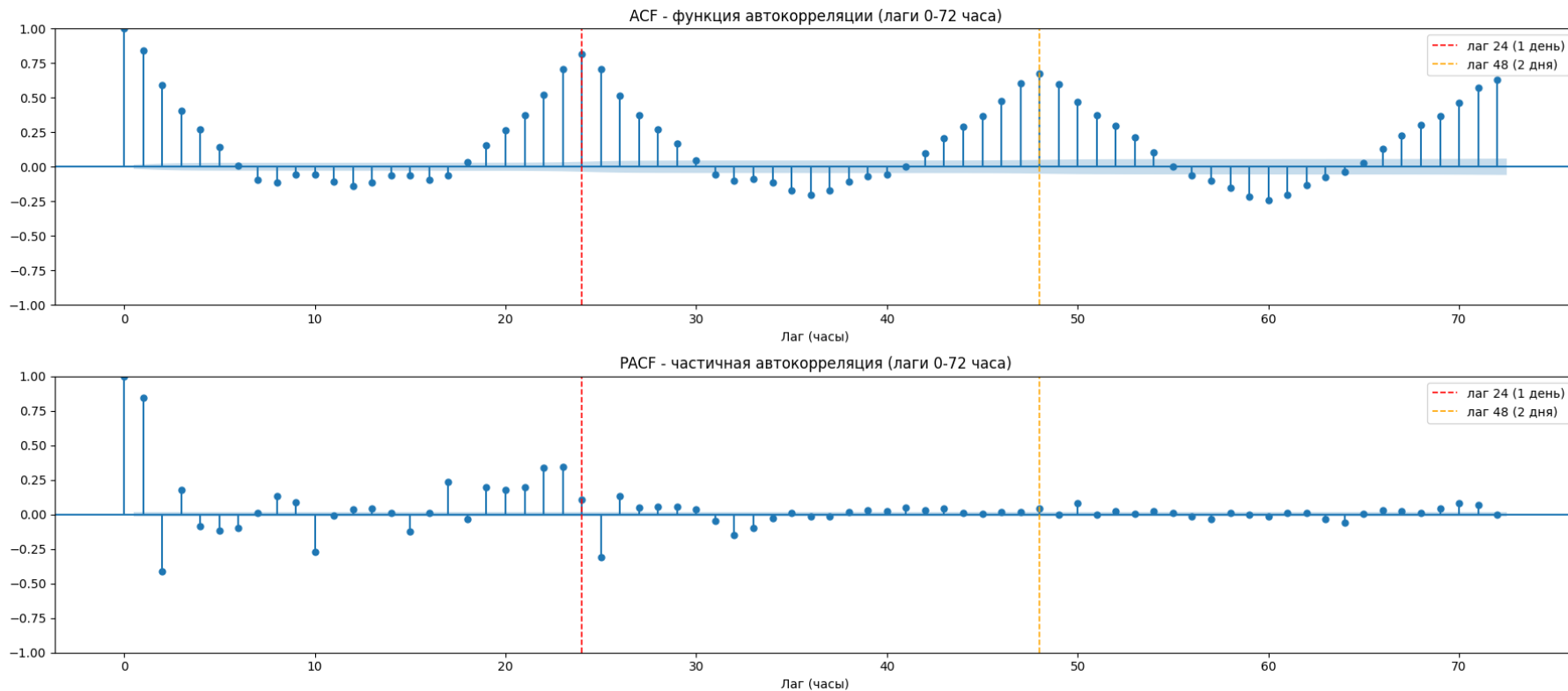


Корреляция признаков

- сильная корреляция между temp и atemp
- корреляция между температурой и кол-вом арендованных велосипедов
- слабая отрицательная корреляция между hum и cnt



Датасет – временной ряд



Кодирование и удаление признаков

Удаления:

- atemp удалён - корреляция с temp 0.99
- dteday удалён - дата закодирована по отдельности

Кодирования:

- ONE для weathersit и season
- Дождь и снег (3 и 4) объединены
- Циклическое кодирование hr и mnth (sin/cos)

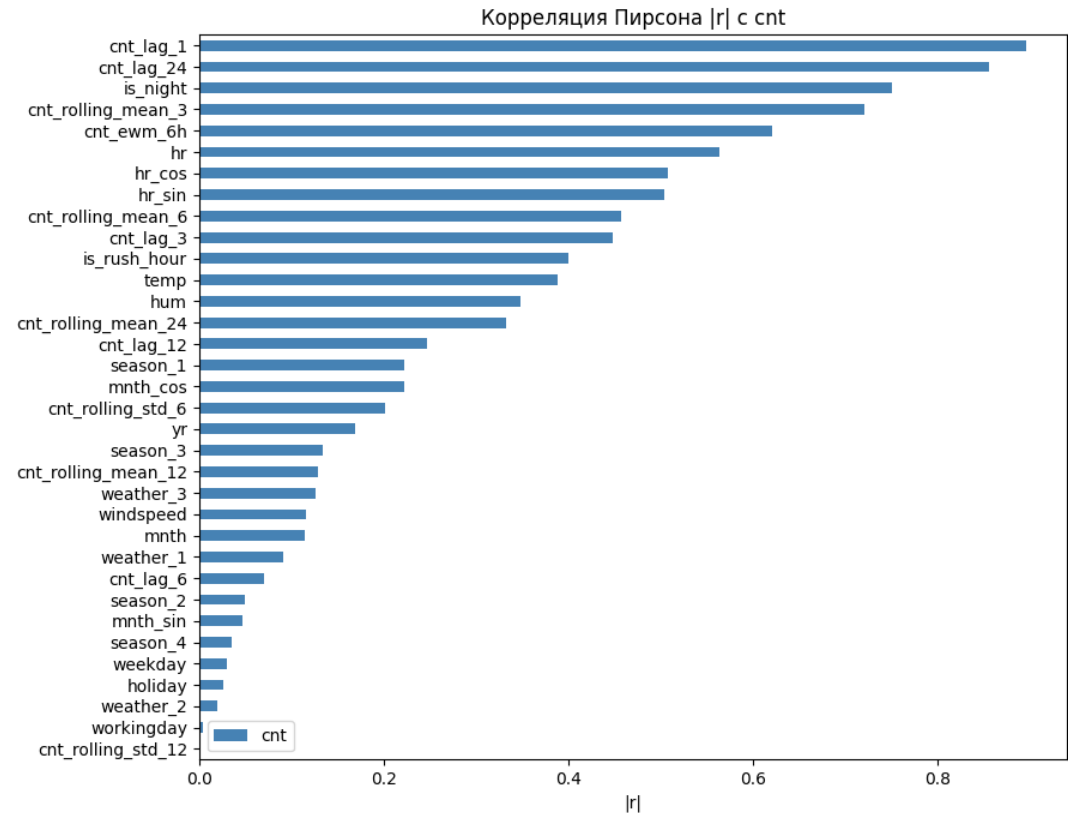
Создание новых признаков

Статические:

- is_rush_hour - (7–9 и 17–19 ч.)
- is_night - (0–5 ч.)

Динамические:

- Лаг (1, 3, 6, 12, 24ч)
- Скользящая средняя (3, 6, 12, 24ч)
- EWM (6ч)



Работа с холодным стартом

Проблема:

- production-сервис может не знать о истории

Решение:

- Обучить модель думать без НИХ

Сценарий	Доля	Доступные lag/rolling
normal	80%	все 12 признаков
partial	15%	cnt_lag_1, cnt_lag_3, cnt_rolling_mean_3, cnt_ewm_6h
cold	5%	ничего (все 12 = NaN)

Разделение данных

Случайный split **недопустим** для временных рядов, произведен хронологический сплит датасета

Сплит	Период	Объем	Обоснование
Train	2011	8645 строк (49.7%)	Обучение модели
Validation	2012 январь–июнь	4358 строк (25.1%)	Подбор гиперпараметров
Test	2012 июль–декабрь	4376 строк (25.2%)	Финальная оценка качества

Baseline модель

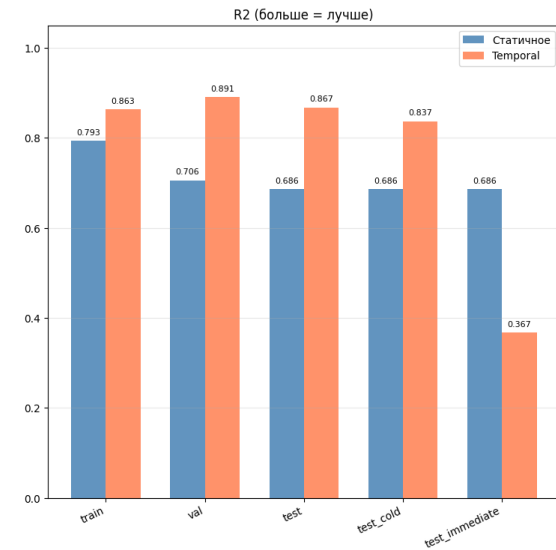
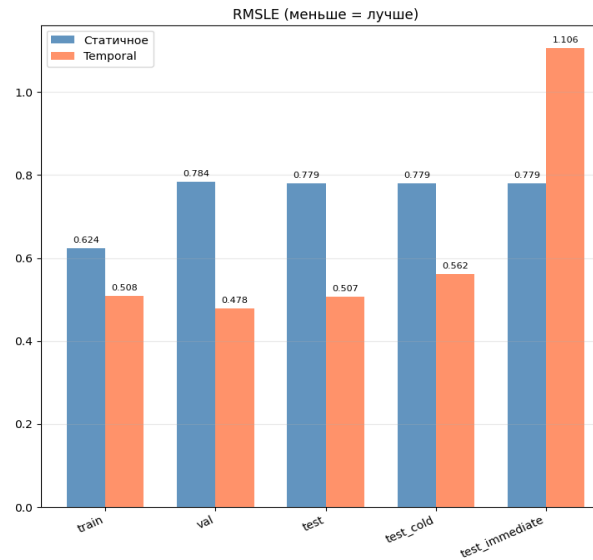
Дерево решений

Дерево 1 (статичное)

- только 22 статичных признака: час, погода, сезон, рабочий день

Дерево 2 (динамичное)

- все 34 признака, включая lag/rolling/EWM.

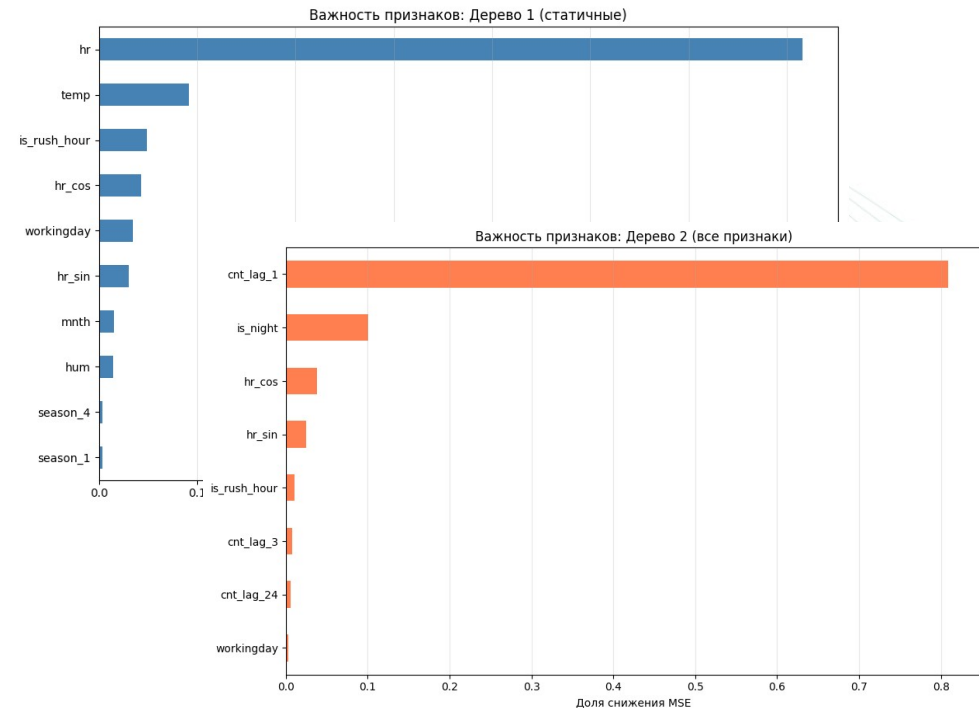


Гипотезы о деревьях

- Дерево 1 даёт идентичные метрики на test, test_cold и test_immediate - Подтверждена
- Дерево 2 значимо превосходит Дерево 1 на чистом test - Подтверждена
- Дерево 2 с медианным заполнением NaN деградирует на test_immediate - Подтверждена

Выводы для CatBoost

- hr доминирует в статичной модели (71.6% FI).
- cnt_lag_1 доминирует в temporal модели (80.8% FI)
- - Влажность hum снижает спрос при $hum > 0.81$
- - is_rush_hour с 4.9% FI



CatBoost модель

Методология обучения

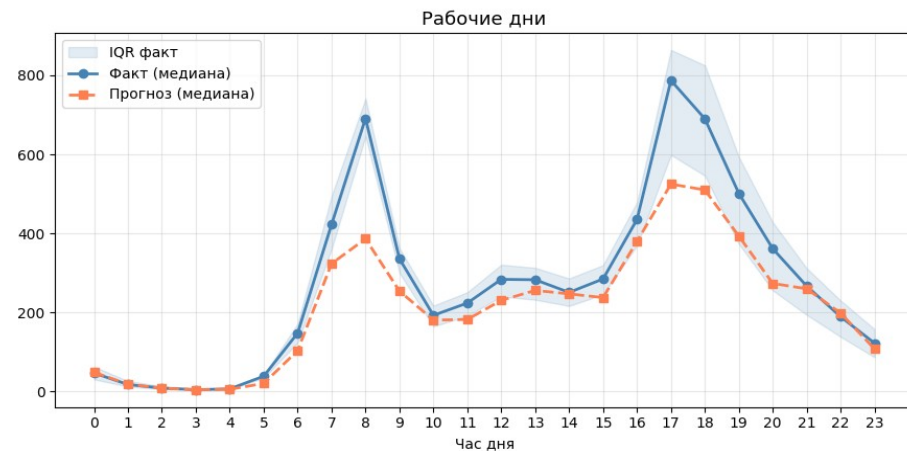
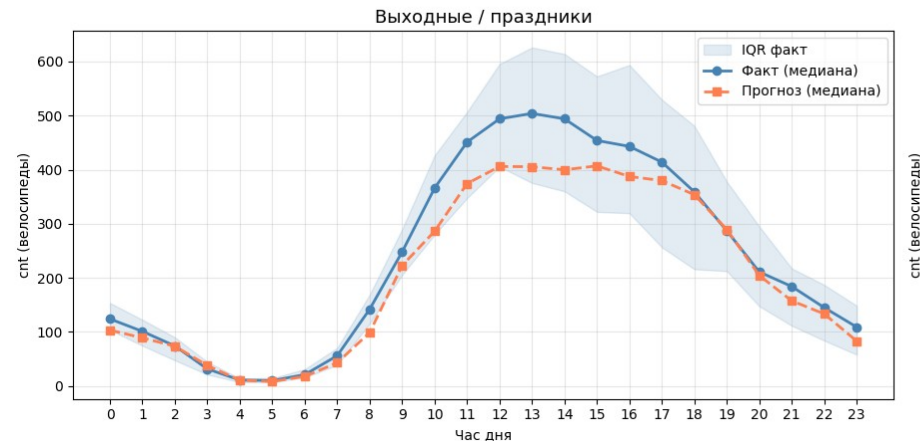
- Train содержит разделения 80/15/5
- Val не содержит пропусков, ищем гиперпараметры через Optuna по rmse
- Test разделен на 3: идеальный, 80/15/5, худший
- Собираем метрики через MLFlow

Подбираем параметры:

- Скорость обучения
- Глубину
- L2 регуляризацию листьев
- Минимальное кол-во данных в листе

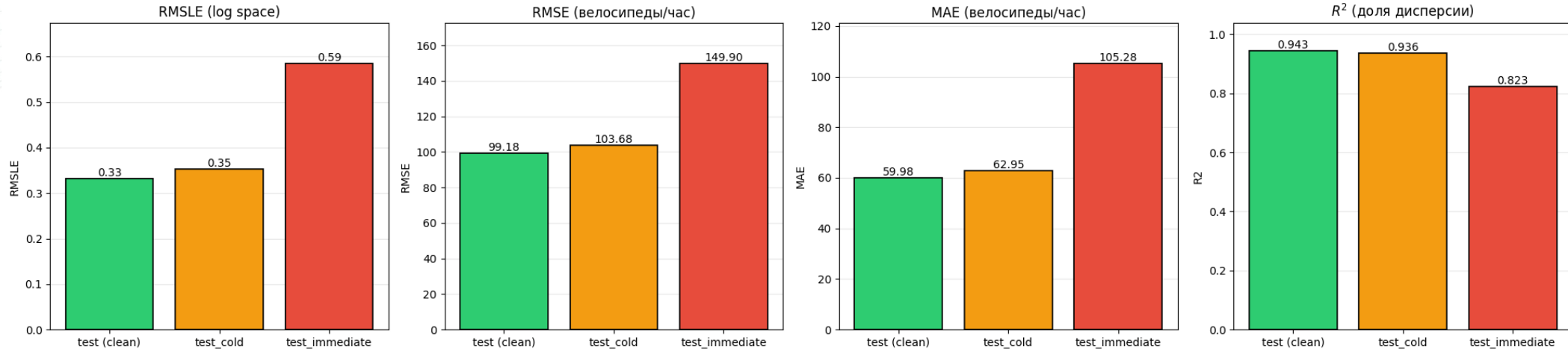
Результаты обучения

Датасет	RMSLE	RMSE	MAE	R ²
val	0.3606	83.1792	51.7248	0.9377
test	0.3289	102.2269	61.3464	0.9441
test_cold	0.3509	106.1801	64.1632	0.9363
test_immediate	0.5779	148.4743	104.5304	0.8273



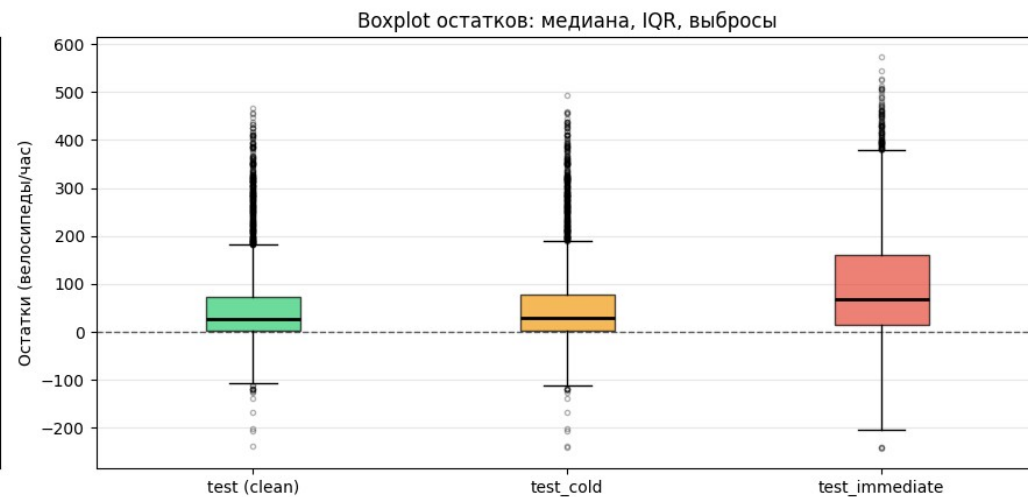
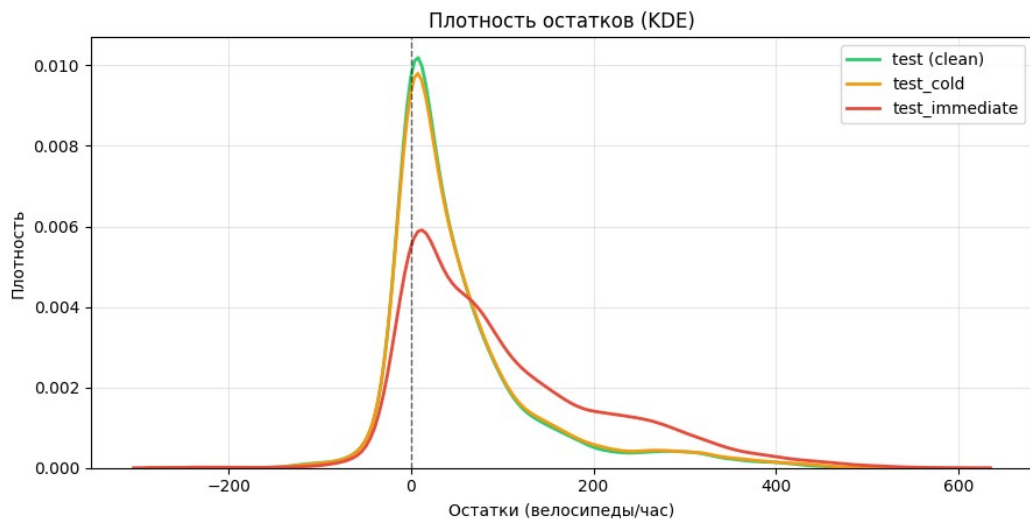
Поведение модели на холодном старте

Сравнение метрик качества по трём режимам test



Распределение остатков

Распределение ошибок: три режима cold start

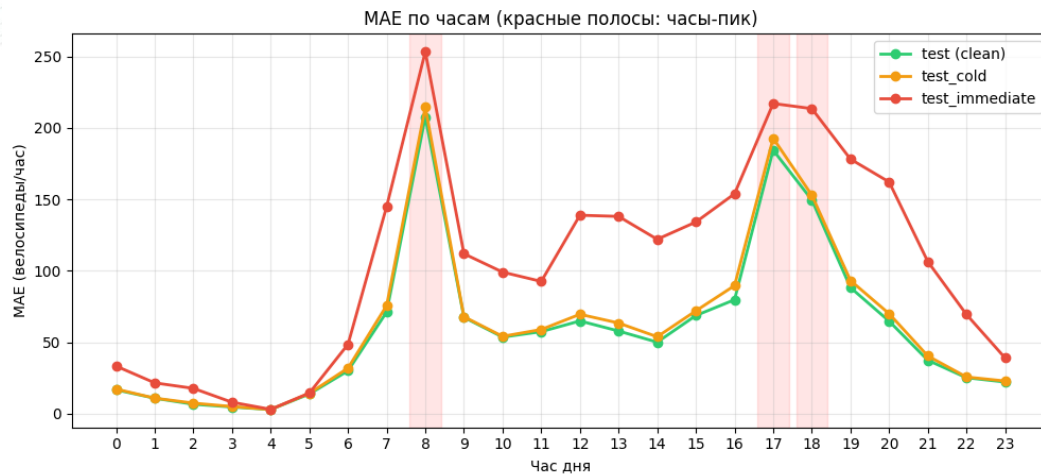


Пики влияния cold start

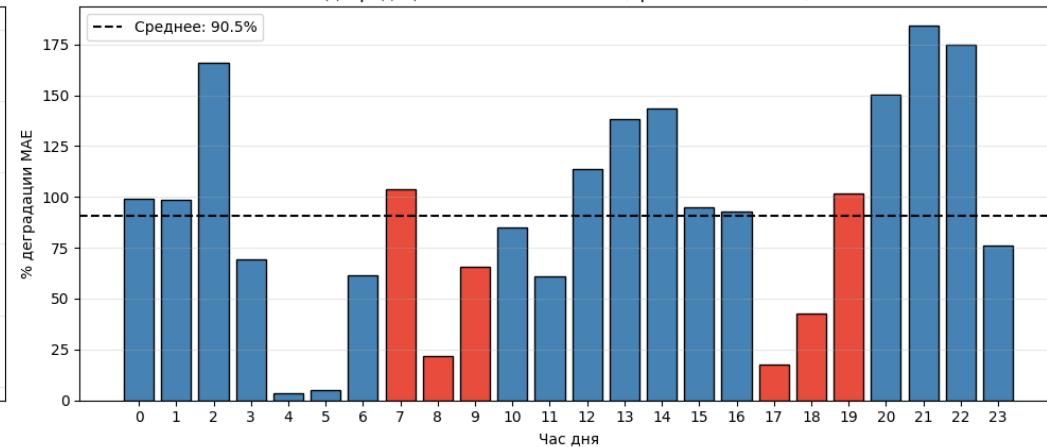
Деградация cold start концентрируется именно в **часы-пик (8:00, 17-18:00)**

- lag несет максимум информации в этот период

Где cold start особенно болезнен



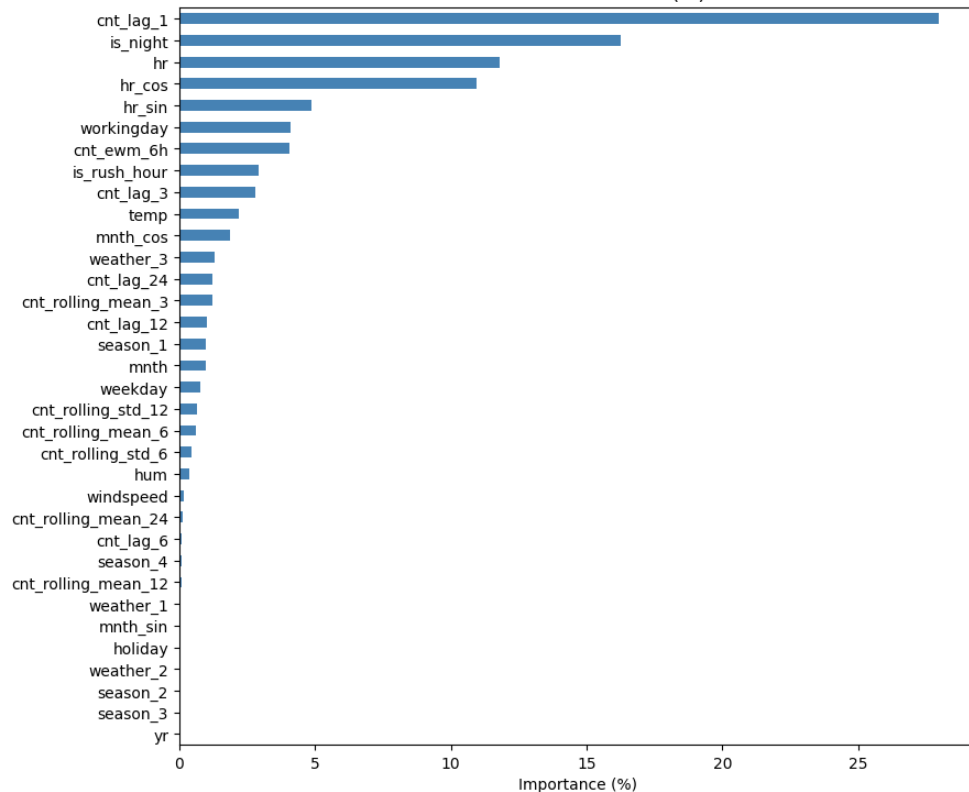
Деградация immediate vs clean (красные: часы-пик)



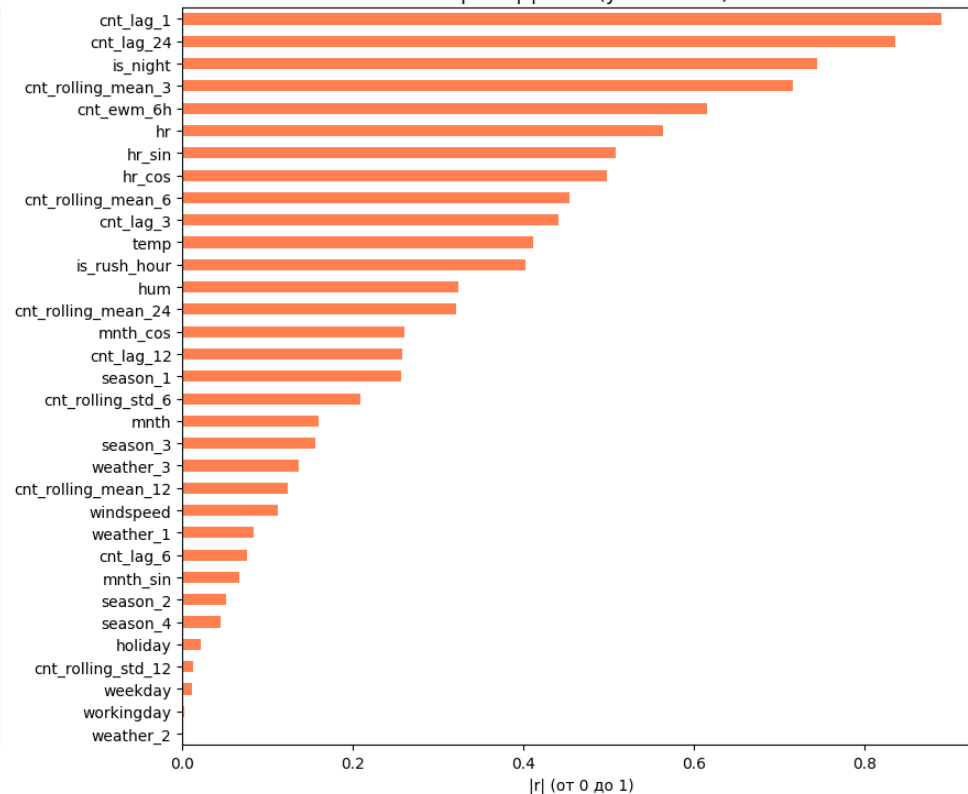
Важность признаков по FI

Важность фич для модели по сравнению с Пирсоном

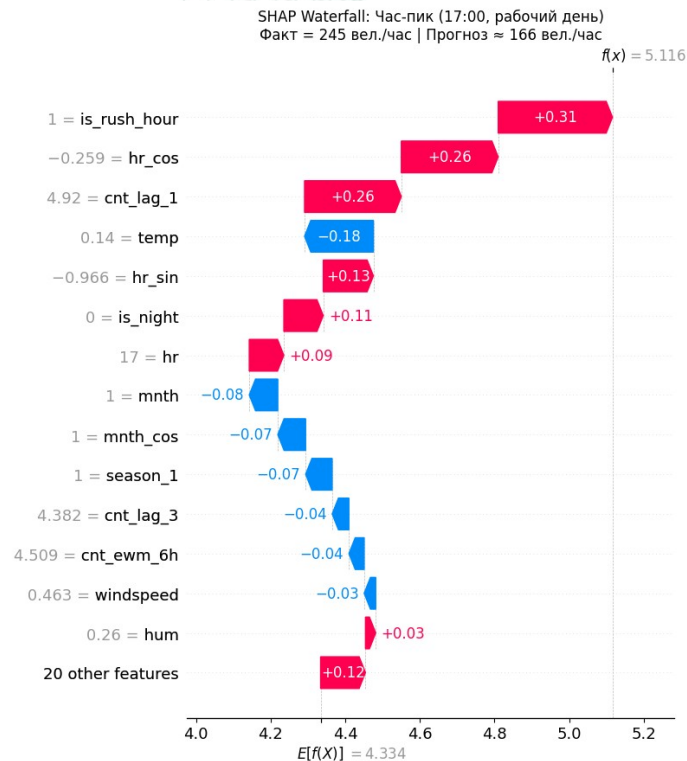
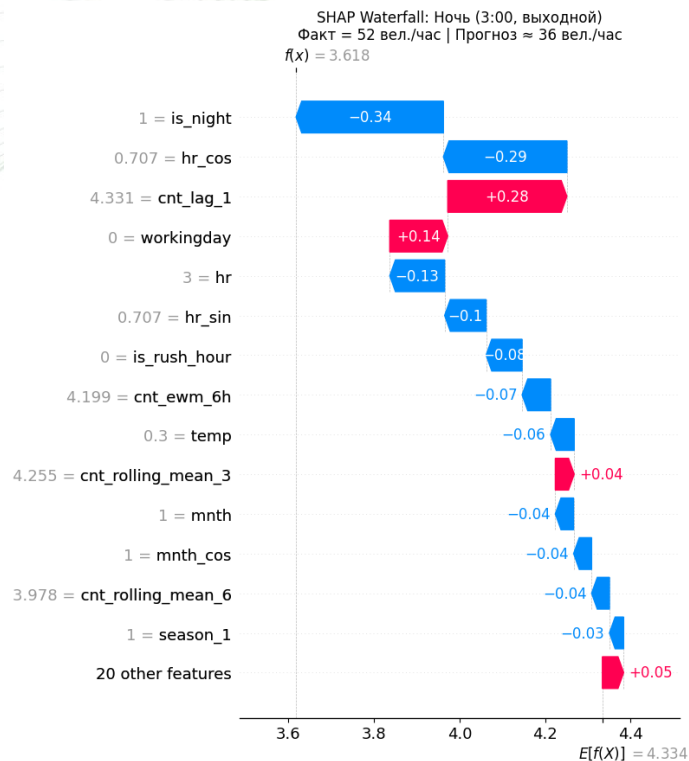
Важность в CatBoost (%)



Пирсон $|r|$ с cnt (yr excluded)

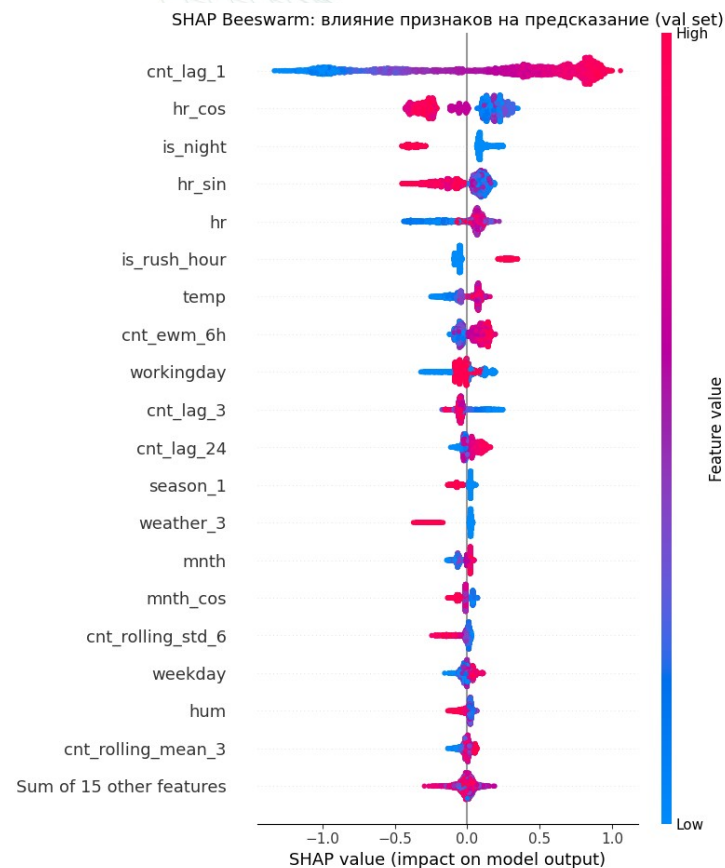


SHAP – интерпритация предсказаний



Выводы по модели

- Модель сильна в "идеальном" режиме
- Модель устойчива к cold start
- Применима для production REST API



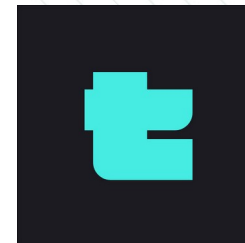
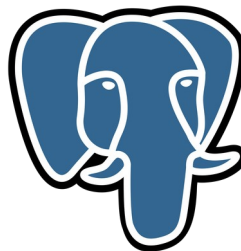
FastAPI Сервис

Стек сервиса

- FastAPI + Uvicorn
- SQLAlchemy + asyncpg
- CatBoost
- Postgres (таблицы для temporal признаков)
- Dependency injector
- Ruff + ty

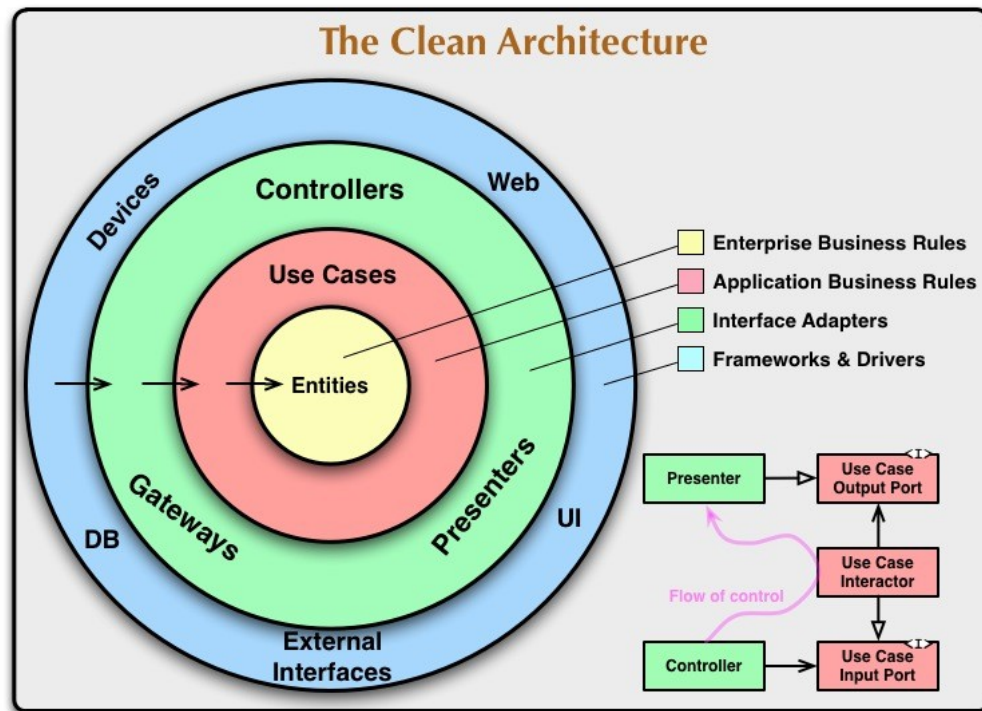


CatBoost



Приложение разбито по слоям:

- Api (controllers)
- Domain
 - Service (use cases)
 - Repository (entities)
- Low (frameworks, DI)



POST **/api/v1/predict** Predict ^

Прогноз почасового спроса на велосипеды.

Parameters Try it out Reset

No parameters

Request body required application/json ▼

Example Value | Schema

```
{
  "dteday": "2011-05-25",
  "hr": 23,
  "holiday": 0,
  "weathersit": 1,
  "temp": 1,
  "hum": 1,
  "windspeed": 1
}
```

Проделанная работа

Код, артефакты, ноутбуки, модели доступны по ссылке или по QR:

- Произведен EDA
- Выполнен Feature Engineering
- Обучены baseline деревья решений
- Обучена финальная CatBoost модель
- Написан веб сервис для инференса модели, с минималистичным api



<https://github.com/kanogame/bikeshare>

Прогнозирование спроса на велопрокат

Иванов Александр Евгеньевич 424-3

Искусственный интеллект. Алгоритмы машинного обучения на языке Python